Guia de Instalação VPS

Setor de Inteligência Artificial

Versão 2.0 - Abril 2026

1. Objetivo deste guia

Este guia ensina a instalar o BKPilot do zero em uma VPS Linux, com foco em Ubuntu 22.04 ou 24.04 LTS.

Ele foi escrito para uso operacional real. Isso significa que não basta instalar pacotes até parar de dar erro. O objetivo é deixar a máquina pronta para:

- rodar o projeto com estabilidade;
- executar Playwright em modo headless;
- operar as CLIs suportadas pelo projeto;
- gerar evidências e artefatos corretamente;
- reduzir erro de configuração manual.

Se você seguir este guia sem pular etapas, ao final terá uma VPS pronta para operar o BKPilot.

2. O que você está instalando

Ao instalar o BKPilot em uma VPS, você não está instalando apenas um projeto Node.

Na prática, está preparando uma máquina para rodar:

- o repositório do projeto;
- as dependências Node.js;
- o browser headless do Playwright;
- conversão de vídeo;

- CLIs de agente;
- geração de evidências e documentação.

Ou seja: a VPS precisa atender tanto a uma camada de automação de browser quanto a uma camada de execução de scripts e geração de artefatos.

3. Visão geral da stack na VPS

Componente	Papel no ambiente
Ubuntu	sistema operacional da VPS
Node.js	runtime principal do projeto
npm	instalação de dependências e CLIs
Git	clonagem e atualização do repositório
Playwright	automação de browser
Chromium	browser controlado pelo Playwright
ffmpeg	conversão de `.webm` para `.mp4`
Claude Code / Codex / OpenCode	ambientes operacionais possíveis

4. Requisitos mínimos da VPS

Recurso	Mínimo	Recomendado
CPU	2 vCPUs	4 vCPUs
RAM	4 GB	8 GB
Disco	20 GB SSD	40 GB SSD
Sistema operacional	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS

Recurso	Mínimo	Recomendado
Rede	saída para internet	saída estável para APIs e sistemas alvo

4.1 Leitura prática desses números

O mínimo serve para rodar o projeto.

O recomendado serve para rodar o projeto com margem mais segura quando:

- o Chromium headless estiver aberto;
- houver geração de vídeo;
- múltiplos processos Node estiverem ativos;
- o QA estiver fazendo mais de uma validação ou rebuild no mesmo ambiente.

5. Acesso inicial e usuário de operação

Evite operar o projeto como `root`. O ideal é criar um usuário dedicado.

5.1 Conectar via SSH

```
ssh root@<IP_DA_VPS>
```

5.2 Criar usuário dedicado

```
 adduser bkpilot
usermod -aG sudo bkpilot
su - bkpilot
```

5.3 Por que isso importa

Rodar como `root` até pode parecer mais simples no início, mas aumenta risco operacional. Um ambiente de automação já tem poder suficiente; não vale ampliar isso desnecessariamente.

5.4 Hardening mínimo da VPS

Como esta VPS pode armazenar credenciais, vídeos, screenshots e outros artefatos de cliente, recomenda-se aplicar um endurecimento mínimo logo no início da preparação do ambiente.

Checklist mínimo recomendado:

- ativar o firewall liberando o SSH antes;
- instalar e validar o `fail2ban`;
- operar com usuário dedicado, não com `root`;
- proteger arquivos de segredo com permissões restritas;
- testar o acesso com o usuário operacional antes de endurecer o SSH.

Exemplo de sequência segura:

```
 sudo ufw allow OpenSSH
sudo ufw enable
sudo apt install -y fail2ban
sudo systemctl enable fail2ban
sudo systemctl start fail2ban
chmod 600 .env 2>/dev/null || true
```

Se a operação evoluir para uso contínuo com dados sensíveis, avance depois para:

- autenticação SSH por chave;
- desativação de login por senha;
- desativação de login direto como `root`;
- backup criptografado dos artefatos relevantes.

6. Atualizar o sistema

Antes de instalar qualquer dependência do projeto, atualize os pacotes do sistema.

```
 sudo apt update
sudo apt upgrade -y
```

Verificação

```
cat /etc/os-release
```

Você deve confirmar que a VPS realmente está em Ubuntu 22.04 ou 24.04.

7. Instalar dependências básicas do sistema

```
sudo apt install -y \
 curl \
 wget \
 unzip \
 build-essential \
 ca-certificates \
 gnupg \
 lsb-release \
 software-properties-common \
 git
```

O que esse bloco resolve

Esse conjunto cobre a base de trabalho:

- download de pacotes;
- compilação de dependências nativas;
- uso de repositórios externos;
- controle de versão.

Sem isso, a instalação do restante tende a quebrar de forma fragmentada.

8. Instalar Node.js

O projeto roda sobre Node.js. Use uma versão LTS moderna.

8.1 Instalar via NodeSource

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
```

```
sudo apt install -y nodejs
```

8.2 Verificar instalação

```
node -v
npm -v
```

Resultado esperado

Comando	Esperado
`node -v`	`v22.x.x`
`npm -v`	`10.x.x` ou superior

9. Instalar ffmpeg

O BKPilot usa vídeo como evidência. O Playwright grava em `.webm` e o projeto converte para `.mp4` quando `ffmpeg` está disponível.

```
sudo apt install -y ffmpeg
```

Verificação

```
ffmpeg -version
```

Observação importante

Sem `ffmpeg`, o projeto ainda funciona. Mas a evidência fica em `.webm`, o que costuma ser menos conveniente para entrega e revisão.

10. Clonar o repositório

```
cd ~
git clone <URL_DO_REPOSITORIO> bkpilot
```

```
cd bkpilot
```

Substitua `<URL_DO_REPOSITORIO>` pela URL real do repositório.

10.1 Checkpoint antes de instalar dependências

Antes de rodar qualquer comando de instalação, confirme que você realmente está dentro da pasta do projeto:

```
pwd
ls
```

Você deve ver o diretório `bkpilot` no caminho atual e arquivos como `package.json`, `setup.sh` e a pasta `clients/`.

Se `package.json` não aparecer, pare e volte para a pasta correta antes de continuar. Isso evita instalar dependências fora do repositório, por exemplo diretamente em `/root`.

11. Instalar dependências do projeto

```
npm install
```

Como saber se deu certo

Depois do `npm install`, valide que a instalação aconteceu dentro do repositório e que os pacotes principais ficaram disponíveis:

```
ls -d node_modules
ls package-lock.json
npm ls playwright xlsx docx canvas dotenv --depth=0
```

Se quiser um teste direto de carregamento dos pacotes principais:

```
node -e "require('playwright'); require('xlsx'); require('docx'); require('dotenv');
```

Se `package.json` não estiver na pasta atual ou se `npm install` criar arquivos no lugar errado, apague esses artefatos indevidos e volte para o diretório correto antes de prosseguir.

O que isso instala

Pacote	Papel
`playwright`	automação de browser
`xlsx`	leitura e escrita de planilhas
`docx`	geração de documentos Word
`canvas`	gráficos e suporte visual
`dotenv`	leitura de variáveis de ambiente

Se `canvas` falhar

Algumas VPS exigem libs nativas extras.

```
sudo apt install -y libcairo2-dev libjpeg-dev libpango1.0-dev libgif-dev librsvg2-dev
npm install
```

12. Instalar dependências do browser

O Chromium headless precisa de bibliotecas de sistema que nem sempre vêm por padrão na VPS.

```
sudo npx playwright install-deps chromium
```

Esse passo é crítico. Sem ele, o browser pode até instalar, mas falhar ao abrir.

13. Instalar o Chromium do Playwright

```
npx playwright install chromium
```

Teste rápido

```
node -e "const { chromium } = require('playwright'); (async()=>{ const b = await chro
```

Se aparecer `Chromium OK`, a base do browser está funcional.

14. Instalar as CLIs operacionais

O BKPilot hoje é multi-CLI. Isso significa que a VPS pode ser preparada para mais de um ambiente operacional.

14.1 Claude Code

```
npm install -g @anthropic-ai/claude-code
```

Verificação:

```
claude --version
```

14.2 Codex

```
npm install -g @openai/codex
```

Verificação:

```
codex --version
```

14.3 OpenCode

```
npm install -g opencode-ai
```

Verificação:

```
opencode --version
```

14.4 Qual delas é obrigatória

Depende da operação da equipe.

Se o time opera majoritariamente em uma CLI, instale ao menos a principal. Se a VPS for ambiente multiuso do time, faz sentido instalar as três.

15. Configurar os arquivos `.env`

O BKPilot separa credenciais em dois níveis:

- **Raiz (`.env`)** — integrações globais: Jira, GitHub Issues
- **Por cliente (`clients/<id>/.env`)** — senha do usuário de QA do cliente

15.1 `.env` raiz (integrações globais)

Antes de copiar, vale conferir os arquivos ocultos do diretório:

```
ls -la
```

Depois execute um comando por vez:

```
cp .env.example .env
nano .env
```

Preencha apenas se for usar integração com issue tracker:

```
JIRA_URL=https://suaempresa.atlassian.net
JIRA_EMAIL=email@empresa.com
JIRA_TOKEN=...
JIRA_PROJECT_KEY=BUG

GITHUB_REPO=org/repositorio
GITHUB_TOKEN=...
```

Se não for usar integrações externas agora, deixe o arquivo como está e siga em frente.

15.2 `.env` por cliente (credenciais de QA)

Cada cliente tem seu próprio arquivo de credenciais isolado. A forma recomendada é o script `novo-cliente.sh`, que cria a estrutura inteira do cliente (pasta, `.env`, `config.json` e `login.js` skeleton) de uma vez:

```
./novo-cliente.sh <id> --nome "Nome do Cliente" --url https://app.cliente.com.br
nano clients/<id>/.env   # preencher QA_PASSWORD
```

Alternativamente, manualmente:

```
cp clients/.env.example clients/<id>/.env
nano clients/<id>/.env
```

Preencha com a senha do usuário de QA do ambiente do cliente:

```
QA_PASSWORD=<senha_do_usuario_qa>
```

O nome da variável deve corresponder ao campo `envPassword` definido em `clients/<id>/config.json`.

15.3 Sobre autenticação das CLIs

Nesta operação, o **Claude Code** será autenticado pelo próprio fluxo de login da CLI, e não por `ANTHROPIC_API_KEY` no `.env` do projeto:

```
claude   # solicita login na primeira execução
```

Em outras palavras: para o fluxo padrão do BKPilot com Claude Code, não é necessário configurar `ANTHROPIC_API_KEY` no `.env` do projeto.

As demais CLIs devem seguir o método de autenticação exigido por cada ferramenta e pelo plano contratado pela equipe.

15.4 Login do Claude Code em VPS remota

Quando o login do Claude Code ocorrer em uma VPS acessada por SSH, o fluxo pode depender do navegador do seu computador local.

Procedimento recomendado:

```
claude
```

Se a CLI exibir uma URL de autenticação:

- abra essa URL no navegador do seu computador local;
- conclua o login com a conta autorizada da empresa;
- se o navegador exibir um código, cole esse código de volta no terminal da VPS;
- se aparecer erro de código inválido, repita o fluxo e tente concluir sem demora, porque o código pode expirar.

Antes de endurecer o SSH da VPS, vale concluir esse login uma vez e confirmar que a sessão da CLI ficou funcional no usuário operacional correto.

Regra importante

Nunca coloque senha inline no comando de execução. A senha deve vir do `clients/<id>/.env`.

16. Executar o setup do projeto

```
chmod +x setup.sh
bash setup.sh
```

O que o `setup.sh` faz

Etapa	O que faz
Pré-requisitos	Verifica Node.js, npm, Claude Code e ffmpeg
Playwright MCP	Garante que o MCP está disponível via `npx`
Chromium	Instala o browser headless do Playwright
Pastas	Cria `clients/<id>/estado/screenshots/`, `clients/<id>/resultado/`, `cenarios/`, `clients/`

Etapa	O que faz
`.env` raiz	Copia `.env.example` → `.env` se não existir (apenas integrações globais — Jira, GitHub)

O que o `setup.sh` não faz

O `setup.sh` é **bootstrap de máquina** — roda uma vez por VPS, não por cliente. Ele **não configura o `.env` por cliente** porque cada cliente tem credenciais diferentes e o setup não tem como saber quais clientes serão operados nesta VPS.

Para cada cliente novo, use o script dedicado `novo-cliente.sh` (recomendado), que cria a estrutura completa do client pack:

```
./novo-cliente.sh <id> --nome "Nome do Cliente" --url https://app.cliente.com.br
nano clients/<id>/.env   # preencher QA_PASSWORD
```

Ou manualmente:

```
cp clients/.env.example clients/<id>/.env
nano clients/<id>/.env   # preencher QA_PASSWORD
```

Sobre Claude Code

O `setup.sh` valida que `claude` está instalado. Se o comando não for encontrado, instale antes de continuar:

```
npm install -g @anthropic-ai/claude-code
```

Após instalar, execute `claude` uma vez para concluir o login pela CLI antes de rodar o setup novamente.

17. Rodar o Skill Converter

O BKPilot mantém as skills em `src/*.md` como fonte canônica e usa o `converter/` para gerar distribuições equivalentes para as CLIs suportadas.

```
node converter/render.js --lint
node converter/render.js --build-all
```

O que validar aqui

- as skills estão consistentes com a fonte canônica em `src/`;
- os targets são gerados sem erro;
- a instalação não quebrou a cadeia de build do projeto.

Relação com a estrutura de clientes

O Skill Converter opera sobre as **skills genéricas** — ele não lida com configuração de clientes. A lógica específica de cada cliente (credenciais, flows, seletores) vive exclusivamente em `clients/<id>/` e é carregada em tempo de execução pelo `core/client.js`.

Onde vive	O que contém
`src/`	Fonte canônica das skills
`converter/`	Geração de distribuições para as CLIs suportadas
`.claude/commands/`	Projeção operacional para Claude Code
`dist/codex/`	Distribuição para Codex
`dist/opencode/`	Distribuição para OpenCode
`clients/<id>/`	Configuração, login e flows do cliente
`clients/<id>/.env`	Credenciais do cliente (nunca

Onde vive	O que contém
	commitadas)
`clients/<id>/entregaveis/automacao/<stack>/`	Pacotes de automação destinados ao cliente
`clients/<id>/clients/<id>/resultado/<timestamp>/governanca/`	Artefatos internos de governança, não enviados automaticamente ao cliente

Isso significa que adicionar ou alterar um cliente **não requer rodar o converter** — basta criar ou editar a pasta do cliente.

18. Teste operacional mínimo

Depois da instalação, valide o ambiente com uma sequência curta:

```
node -v
npm -v
git --version
ffmpeg -version
npx playwright --version
```

Depois valide apenas a(s) CLI(s) realmente instalada(s) para essa VPS:

```
claude --version
# ou
codex --version
# ou
opencode --version
```

E faça o teste de browser:

```
node -e "const { chromium } = require('playwright'); (async()=>{ const b = await chro
```

19. Primeiro uso real

Depois de validar o ambiente, autentique e abra a CLI escolhida pelo time:

```
claude
```

Ou a CLI escolhida pela equipe.

Antes de iniciar um fluxo real, confirme que:

- a autenticação da CLI foi concluída;
- a sessão ficou persistida no usuário operacional correto;
- o browser headless já foi validado no passo anterior.

A partir daí, o primeiro fluxo recomendado continua sendo:

```
/explorar <URL> --login <email>
```

Esse primeiro teste é importante porque valida o caminho completo:

- CLI;
- projeto;
- browser;
- credencial;
- artefatos.

20. Checklist final

Item	Como verificar	Esperado
Ubuntu atualizado	`cat /etc/os-release`	22.04 ou 24.04
Node.js	`node -v`	`v22.x.x`
npm	`npm -v`	versão instalada
Git	`git --version`	versão instalada

Item	Como verificar	Esperado
ffmpeg	`ffmpeg -version`	versão instalada
Playwright	`npx playwright --version`	versão instalada
Chromium	teste headless	`Chromium OK`
`.env` raiz	`ls -la .env`	arquivo presente com permissão controlada
`.env` por cliente	`ls -la clients/<id>/.env` e `grep '^QA_PASSWORD=' clients/<id>/.env`	arquivo presente e variável preenchida
Permissões dos segredos	`ls -l .env clients/<id>/.env`	preferencialmente restritas ao usuário operacional
UFW	`sudo ufw status`	ativo com SSH liberado
fail2ban	`sudo fail2ban-client status sshd`	jail `sshd` ativa
Skill Converter	`node converter/render.js --lint`	sem erro
CLI operacional	`claude --version` ou `codex --version` ou `opencode --version`	disponível conforme a CLI instalada

21. Problemas comuns

21.1 `canvas` não instala

```
sudo apt install -y libcairo2-dev libjpeg-dev libpango1.0-dev libgif-dev librsvg2-dev
npm rebuild canvas
```

21.2 Chromium não abre

```
sudo npx playwright install-deps chromium
```

Se continuar falhando, revise bibliotecas do sistema e teste o browser headless isoladamente.

21.3 CLI não reconhecida

Se `claude`, `codex` ou `opencode` não forem encontrados, confirme:

- instalação global via npm;
- `PATH` do usuário correto;
- shell atual após a instalação.

21.4 `.env` ausente ou incompleto

Se a autenticação falhar logo no começo, verifique os dois níveis sem expor o conteúdo inteiro na tela:

```
ls -la .env
ls -la clients/<id>/.env
grep '^QA_PASSWORD=' clients/<id>/.env
```

O erro `QA_PASSWORD ausente em clients/<id>/.env` indica que o arquivo do cliente não foi criado. Corrija com:

```
cp clients/.env.example clients/<id>/.env
nano clients/<id>/.env
```

21.5 Projeto instala, mas não opera

Instalar dependências não significa que a cadeia operacional está saudável. Se o projeto abre, mas não executa bem, valide na ordem:

1. browser headless;
2. `.env`;
3. skill converter;
4. CLI;
5. primeiro fluxo real com `/explorar`.

21.6 Rodei `npm install` fora do projeto

Se você rodou `npm install` fora da pasta do BKPilot, normalmente verá `node_modules`, `package.json` ou `package-lock.json` em um local inesperado, como `/root`.

Valide com:

```
pwd
ls
```

Se estiver no lugar errado, apague os artefatos indevidos, volte para a pasta do repositório e repita a instalação no diretório correto.

21.7 Juntei dois comandos e criei um arquivo estranho

Se você colar dois comandos na mesma linha ou digitar algo como `cp ... nano ...` sem separação correta, pode acabar criando um arquivo com nome inesperado.

Para localizar melhor esse tipo de arquivo:

```
ls -la
ls -lb
ls -li
```

Se o nome estiver muito estranho, use o inode com `find ... -inum ... -delete` para remover com segurança.

21.8 Não estou vendo `.env` ou `.env.example`

Arquivos iniciados com ponto não aparecem no `ls` simples.

Use:

```
ls -la
```

Isso evita concluir incorretamente que o template não existe.

22. Transferência de artefatos para o computador local

Como o BKPilot pode gerar vídeos, screenshots, logs, pacotes de automação e outros artefatos, defina desde cedo uma forma segura de tirar esses arquivos da VPS.

Opções práticas:

- usar o gerenciador de arquivos/SFTP do cliente SSH;
- compactar a pasta da execução ou o pacote de automação com `tar -czf`;
- baixar via `scp`;
- sincronizar recorrências com `rsync`.

Exemplo de compactação de uma execução:

```
tar -czf cliente-01_execucao-001.tar.gz clients/cliente-01/clients/<id>/resultado/202
```

Exemplo de compactação de um pacote de automação para cliente:

```
mkdir -p exports
tar -czf exports/cliente-01_playwright-ts_automacao.tar.gz \
  entregaveis/cliente-01/automacao/playwright-ts
```

Exemplo de cópia via `scp` a partir do computador local:

```
scp usuario@IP_DA_VPS:/home/bkpilot/bkpilot/exports/cliente-01_playwright-ts_automaca
```

Antes de compactar um pacote de automação, confirme que todos os relatórios `.md` destinados ao cliente possuem `.pdf` correspondente. Essa é uma regra obrigatória das skills `/gerar-automacao-cliente` e `/auditar-automacao-cliente`.

Não inclua automaticamente `clients/<id>/clients/<id>/resultado/<timestamp>/governanca/` no pacote enviado ao cliente. Essa pasta pode conter metadados internos, autoria da execução e informações de governança.

Quando os artefatos tiverem dado sensível de cliente, baixe, arquive e remova da VPS o que não precisar permanecer nela.

23. Boas práticas para VPS de operação

- use usuário dedicado;
- não rode rotina diária como `root`;
- mantenha o sistema atualizado;
- não armazene credenciais fora de `clients/<id>/.env` (QA) ou `.env` raiz (integrações);
- aplique permissões restritas aos arquivos de segredo, preferencialmente `chmod 600`;
- valide browser e CLI após qualquer atualização importante;
- mantenha espaço em disco sob observação, porque vídeos e screenshots crescem rápido.

24. Conclusão

Uma instalação boa do BKPilot em VPS não é só conseguir rodar `npm install`. É deixar o ambiente pronto para operar com consistência.

Se a VPS estiver corretamente preparada, você reduz:

- tempo perdido em troubleshooting básico;
- falhas de browser;
- divergência entre ambientes;
- risco de artefato incompleto;
- erro operacional do QA.

Em caso de dúvida, prefira validar cada camada em ordem: sistema, Node, browser, projeto, converter, CLI e fluxo real.

BKPilot - Setor de Inteligência Artificial